

System Architecture

Intelligent Video Analytics Platform

The platform follows a linear tracking-by-detection pipeline: each video frame flows through eight stages, from raw capture to a final saved report. All three of this project's frontends (CLI, Streamlit, Desktop) run the identical pipeline below via one shared engine, `analytics_core.py` — this is a deliberate design choice explained in Section 2.

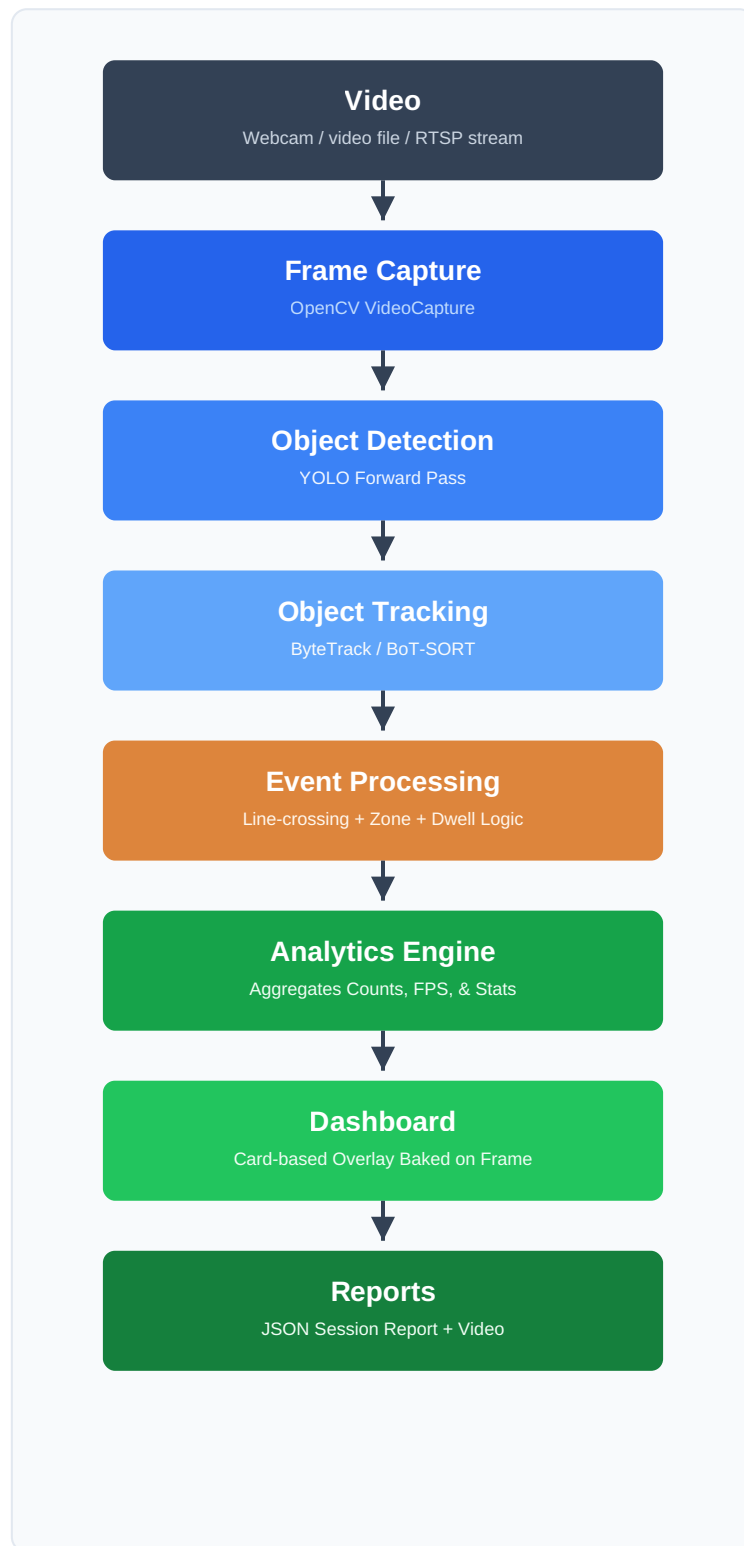


Figure 1 — End-to-end pipeline, one frame's journey from capture to report

1. Pipeline Components

Video `open_capture()`

The input source: a live webcam, an uploaded video file, or an RTSP stream. Webcam sources use DirectShow on Windows rather than the default backend, plus a few throwaway warmup reads — both added after real hardware testing showed the default backend intermittently failing to grab frames right after opening.

Frame Capture `cv2.VideoCapture.read()`

Pulls one frame at a time from the source. In the Streamlit frontend this runs in a background thread (`ThreadedWebcam`) that always keeps only the most recent frame, so a slow consumer (detection + rendering) never falls behind a growing backlog of buffered frames — the fix for an early lag/stutter issue.

Object Detection `model.track(imgsz, device, ...)`

A YOLO forward pass locates objects in the frame, producing bounding boxes, class labels, and confidences. `imgsz` (inference resolution), `device` (CPU/GPU), and `fp16` (half-precision) are all runtime-configurable — measured directly to trade speed against accuracy.

Object Tracking `tracker=bytetrack.yaml / botsort.yaml`

Ultralytics runs ByteTrack or BoT-SORT immediately after detection, in the same `.track()` call. This assigns a persistent ID to each detected object and carries it across frames using motion prediction (Kalman filter) and, for BoT-SORT, camera motion compensation — turning independent per-frame detections into continuous tracked trajectories.

Event Processing `LineZone / PolygonZone .trigger()`

Given this frame's tracked detections, this stage decides what happened: did any object's center cross the counting line (LineZone, CENTER anchor for accuracy)? Is it inside an ROI zone (PolygonZone, bottom-center anchor)? It also updates each track's trajectory buffer and dwell timer here.

Analytics Engine `process_frame() stats dict`

Aggregates everything Event Processing produced into a single stats snapshot for this frame: current/total object counts, IN/OUT totals, per-zone occupancy and longest dwell, running average confidence, and FPS/processing time — this is the data both the Dashboard and Reports stages consume.

Dashboard `_draw_dashboard()`

Renders the Analytics Engine's stats as a card-based overlay baked directly onto the frame — color-coded by performance (green/amber/red for FPS and confidence) — so it's visible live in every frontend and preserved automatically in any recorded video, not just shown in a UI widget that recording wouldn't capture.

Reports `build_report()` / `create_video_writer()`

At the end of a session: `build_report()` writes a JSON summary (counts, FPS, dwell times) to `reports/`, and if recording was enabled, the fully-annotated video was already being written frame-by-frame via a codec-fallback-safe VideoWriter (MJPG → MP4 → XVID → DIVX) to `output/`.

2. Why One Shared Engine, Three Frontends

All eight pipeline stages above live in exactly one place — `analytics_core.py`'s `AnalyticsCore` class — not duplicated across `app.py`, `streamlit_app.py`, and `desktop_app.py`. Each frontend is a thin wrapper: it opens a video source, calls `core.process_frame(frame)` once per frame, and displays the result its own way (an OpenCV window, a Streamlit image element, or a Qt widget).

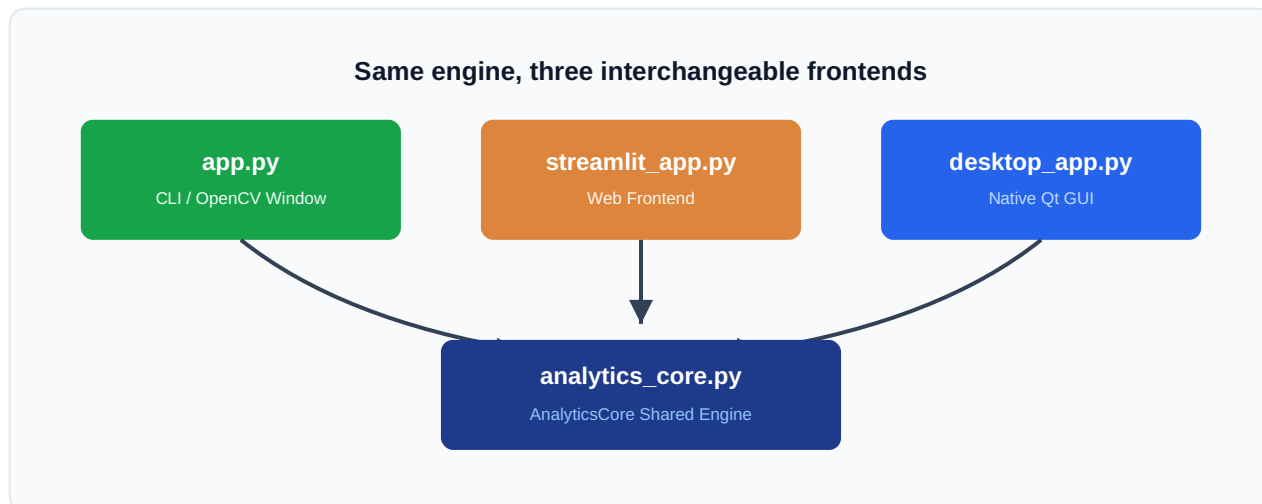


Figure 2 — Three frontends, one engine

This matters beyond code cleanliness: every accuracy fix made during this project (the line-crossing anchor point, the direction-smoothing algorithm, the dwell-time logic) automatically applies to all three frontends at once, and all three are guaranteed to report identical counts on identical input — a real requirement for a technical interview question like *"do your different interfaces agree with each other?"*

3. Design Notes

- **Detection and tracking** are conceptually separate stages but run in a single call (`model.track()`) because Ultralytics internally feeds its own detection output straight into the tracker — there is no reason to run two separate forward passes.
- **The Dashboard's height** is fixed regardless of whether ROI zones are configured, specifically so a video recording's frame size never changes mid-session — an earlier version that resized the dashboard for zones vs. no-zones corrupted recordings when that setting changed between runs.
- **Defensive Engineering:** Every stage from Frame Capture onward is defensive about failure: capture falls back across backends, the video writer falls back across codecs, and FPS values are sanitized against NaN/zero before being used — all three were real bugs hit and fixed during this project.